Python for Engineering

Class 01 - From Mathematical Equation to Algorithm

Notes + examples + working exercises

Class objective: learn to transform a mathematical problem into a logical sequence of inputs, operations, decisions, iterations, and outputs before writing code.

1. The central idea

Solving an equation on paper and programming it are related skills, but they are not identical. Before writing Python, we need to build an **algorithm**: an explicit sequence of steps that transforms inputs into outputs.

$$\boxed{\text{Problem} \rightarrow \text{Algorithm} \rightarrow \text{Code} \rightarrow \text{Validation}}$$

Almost any scientific program combines four structures:

1. **Sequence:** do A, then B, then C.
2. **Decision:** if a condition is true, follow one path; otherwise, follow another.
3. **Iteration:** repeat an operation a given number of times or until a condition is satisfied.
4. **Decomposition:** divide a large problem into smaller problems that will later become functions, modules, or classes.

Working rule: do not start with Python. Start by defining what comes in, what must come out, which equations are involved, and which decisions or repetitions the problem requires.

2. Example 1 - Dynamic pressure

Dynamic pressure is given by:

$$q = \frac{1}{2}\rho V^2$$

Inputs: ρ, V.
Output: q.
Decisions: none for the basic calculation.
Iterations: none.

Algorithm

```
START
Read rho
Read velocity
q = 0.5 * rho * velocity^2
Show q
END
```

Python

```python
rho = 1.225
velocity = 100.0

dynamic_pressure = 0.5 * rho * velocity**2

print(dynamic_pressure)
```

The first logical improvement appears when we ask: *does a negative density make physical sense?* A physical validation rule can become a conditional statement.

3. Example 2 - Quadratic equation

$$ax^2 + bx + c = 0$$

We define the discriminant:

$$\Delta = b^2 - 4ac$$

The mathematics contains three different cases:

$$\Delta > 0, \qquad \Delta = 0, \qquad \Delta < 0$$

Therefore, the algorithm needs **branches**.

```
INPUT: a, b, c
Calculate discriminant

If discriminant > 0:
    calculate two real roots
Else if discriminant == 0:
    calculate one real root
Else:
    report no real roots

OUTPUT: roots or message
```

In Python, these branches are expressed with:

```python
if condition:
    ...
elif another_condition:
    ...
else:
    ...
```

> **Conditionals are not an artificial feature of the language: they represent real branches in mathematical or physical reasoning.**

4. Example 3 - Iterative sum

We want to calculate:

$$S = \sum_{i=1}^{N} i^2$$

For $N = 4$:

$$S = 1^2 + 2^2 + 3^2 + 4^2$$

Now an operation is repeated. We need an accumulator variable:

```
total = 0

for i in range(1, N + 1):
    total = total + i**2
```

After each iteration, conceptually:

$$\text{total}_{new} = \text{total}_{old} + i^2$$

This is especially important in engineering because many simulations have a similar structure:

$$x_{k+1} = x_k + v_k \Delta t$$

$$v_{k+1} = v_k + a_k \Delta t$$

5. Universal template before programming

We will use this template throughout the first weeks:

```
PROBLEM
-------

INPUTS
------

OUTPUTS
-------

EQUATIONS
---------

SPECIAL CASES
-------------

DO I NEED ITERATION?
--------------------

ALGORITHM
---------

VALIDATION
----------
```

This is not bureaucracy: it forces us to separate the mathematical solution from Python syntax.

6. Class 1 exercises

Exercise 1 - Dynamic Pressure Calculator

Implement:

$$q = \frac{1}{2}\rho V^2$$

Inputs: `rho`, `velocity`. Output: `dynamic_pressure`. Think about what should happen if $\rho < 0$.

Exercise 2 - Rocket Equation

Implement Tsiolkovsky's rocket equation:

$$\Delta V = I_{sp} g_0 \ln\left(\frac{m_0}{m_f}\right)$$

Inputs: `Isp`, `g0`, `initial_mass`, `final_mass`. Before programming, identify which physical relationships m_0 and m_f must satisfy.

Exercise 3 - Quadratic Equation Solver

Solve:

$$ax^2 + bx + c = 0$$

Consider:

- $\Delta > 0$: two real roots.
- $\Delta = 0$: one real root.
- $\Delta < 0$: display `No real roots`.

Do not use complex numbers yet.

Exercise 4 - Drag Coefficient

Simplified model:

$$C_D = \begin{cases} 0.3, & M < 0.8 \\ 0.5, & 0.8 \le M < 1.2 \\ 0.25, & M \ge 1.2 \end{cases}$$

Input: Mach. Output: C_D. Draw the logical tree mentally before writing the `if` statement.

Exercise 5 - Sum of Squares

Calculate iteratively:

$$S = \sum_{i=1}^{N} i^2$$

Then validate the result against:

$$S = \frac{N(N+1)(2N+1)}{6}$$

Exercise 6 - Factorial

Implement:

$$N! = N(N-1)(N-2)\cdots 1$$

without using `math.factorial()`.

Exercise 7 - Taylor series of e^x

Approximate:

$$e^x = \sum_{n=0}^{N} \frac{x^n}{n!}$$

Before programming, identify:

- what is repeated;
- what changes at each iteration;
- what value is accumulated;
- what N controls.

Exercise 8 - Vertical motion

System:

$$\dot{h} = v, \qquad \dot{v} = -g$$

Step-by-step approximation:

$$v_{k+1} = v_k - g\Delta t$$

$$h_{k+1} = h_k + v_k \Delta t$$

Repeat while altitude remains non-negative. This will be our first bridge toward dynamic simulation.

7. Mini Project 01 - Ballistic Flight Simulator

First version:

- vertical motion;
- constant gravity;
- no aerodynamic drag;
- fixed time step.

Planned evolution:

Version	Improvement
V1	Constant gravity
V2	Altitude-dependent gravity
V3	Atmospheric density
V4	Aerodynamic drag
V5	Variable mass
V6	Thrust
V7	Rocket trajectory

The purpose is to learn **incremental development**: first build a small, verifiable model, then add complexity in a controlled way.

8. Folder organization for this class

```
Python_Engineering/
‘-- 01_python_bootcamp/
    ‘-- week_01_logic/
        ‘-- class_01_equation_to_algorithm/
            |-- notes/
            |-- exercises/
            |   |-- ex01_dynamic_pressure.py
            |   |-- ex02_rocket_equation.py
            |   |-- ex03_quadratic_solver.py
            |   |-- ex04_drag_coefficient.py
            |   |-- ex05_sum_squares.py
            |   |-- ex06_factorial.py
            |   |-- ex07_taylor_exp.py
            |   ‘-- ex08_vertical_motion.py
            ‘-- mini_project/
                ‘-- ballistic_flight_simulator.py
```

9. First submission for review

Start only with exercises 1, 2, 3, and 4. For each one, first write:

```
INPUTS
OUTPUTS
EQUATIONS
SPECIAL CASES
ALGORITHM
```

Then write the Python code.

The review will follow this order:

Mathematics → Algorithm → Logic → Python → Style → Edge cases

Do not look for the solution before trying. Your own error is useful information: it tells us exactly which part of the reasoning needs reinforcement.

Equation → Algorithm → Code

That is the habit we are going to build throughout the entire program.